

B28: SQL SuperHero Program : Session 8

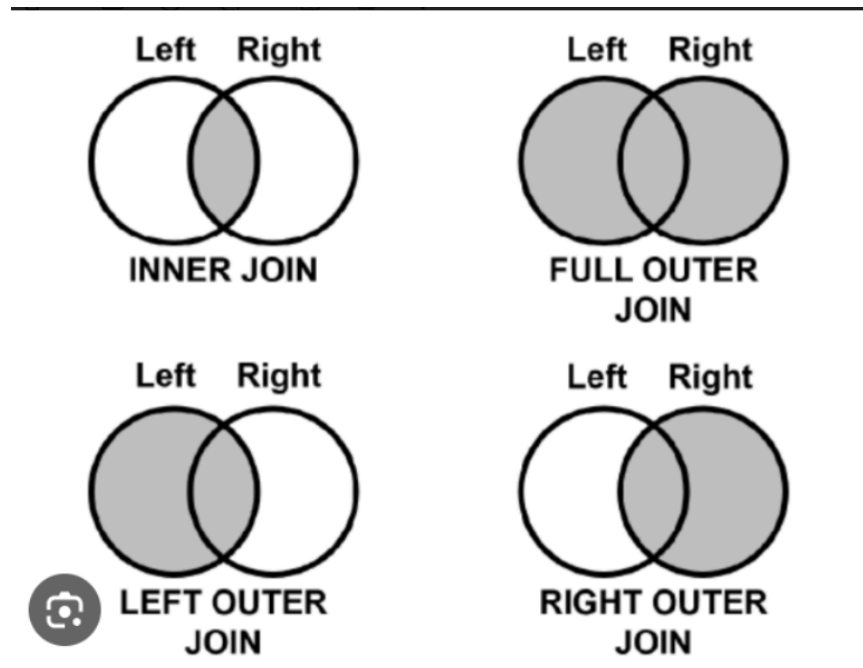
1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

24 February 2025 at 00:17

JOINS

joins are used to combine rows from two or more tables based on a related column between them. Joins allow you to retrieve data from multiple tables in a single query by establishing relationships between the tables.



> Basic funda of total join in a sql is : $N-1$ where N is the number of tables used in query.

Joins:

1. Equi Join/ Inner Join
2. Outer Join --> Right Outer and left Outer
3. Cross Join
4. Self Join

cross join means ; we cross X (multiply) both tables.

if table A has 5 records and table B has 10 records then if we do cross join on A and B , it should be $5 * 10 = 50$ records

```
create table tableA(id Number);
create table tableB(id Number);
```

```
Insert Into TableA Values (1);
Insert Into TableA Values (2);
Insert Into TableA Values (3);
Insert Into TableA Values (4);
```

```
Insert Into TableB Values (4);
Insert Into TableB Values (5);
Insert Into TableB Values (6);
```

```
select * from tableA
```

ID
1
2
3
4

4 rows selected.

```
select * from tableB
```

ID
4
5
6

3 rows selected.

```
select A.id A_id, B.Id B_Id  
from tableA A cross join tableB B
```

A_ID	B_ID
1	4
2	4
3	4
4	4
1	5
2	5
3	5
4	5
1	6
2	6
3	6
4	6

12 rows selected.

$$4 \times 3 = 12$$

If I add a null value in any table.

```
insert into A values (null)
```

1 row(s) inserted.

```
select * from A
```

ID
1
2
3
-

4 rows selected.

→ null added
→ Now Table A has 4 records

Now Table A has 4 records and B has 4 records, so cross join will give you $4 \times 4 = 16$ records.

Equi Join/ inner Join:

It gives records when they fully satisfy the condition.

```
12  
13 select A.id a_id, B.id b_id  
14 from TableA A, TableB B  
15 Where A.Id = B.Id  
16  
17 select A.id a_id, B.id b_id  
18 from TableA A  
19 Inner Join TableB B On A.Id = B.Id;  
20  
21 select * from tableB  
22
```

A_ID	B_ID
4	4

Download CSV

```
select A.id A_id, B.Id B_Id
from tableA A , tableB B
where A.Id = B.Id
```

→ only who exactly match

A_ID	B_ID
4	4

→ 4 in both tables

```
select A.id a_id, B.id b_id
from TableA A
left Join TableB B On A.Id = B.Id;
```

```
38 select A.id a_id, B.id b_id
39 from TableA A
40 left Join TableB B On A.Id = B.Id;
41
```

A_ID	B_ID
4	4
1	-
2	-
3	-

Download CSV

4 rows selected.

```
select A.id a_id, B.id b_id
from TableA A
Right Join TableB B On A.Id = B.Id;
```

```

38 select A.id a_Id, B.id b_id
39 from TableA A
40 Right Join TableB B On A.Id = B.Id;
41 |

```

A_ID	B_ID
4	4
-	6
-	5

[Download CSV](#)

```
select * from tableA
```

ID
1
2
3
4

4 rows selected.

```
select * from tableB
```

ID
4
5
6
1

4 rows selected.

```
select A.Id A_id, B.Id B_id
From TableA A Left join TableB B on A.id = B.Id
```



A_ID	B_ID
4	4
1	1
2	-
3	-

4 rows selected.

who is left → Table A
 so all from left &
 matching from
 Right

```
select A.Id A_id, B.Id B_id
From TableA A Right join TableB B on A.id = B.Id
```



A_ID	B_ID
1	1
4	4
-	6
-	5

4 rows selected.

who is Right → Table B
 So all from Right
 &
 matching from left

- If we mention LEFT/RIGHT, it means we are talking about the left side or right side table.
- LEFT Join means, pick all records from left side table, does not matter if satisfy condition or not
- RIGHT Join means, pick all records from right side table, does not matter if satisfy condition or not

```
select A.id , B.Id  
from tableA A Right join tableB B on A.id = B.Id
```

ID	ID
4	4
-	6
-	5

3 rows selected.

same Right join
but different o/p.



```
select A.id , B.Id  
from tableA A Right join tableB B on A.id = B.Id
```

ID	ID
4	4
-	6
-	5

3 rows selected.

bcz left & right is
just position

```
create table test1 (id number);  
insert into test1 values (1);  
insert into test1 values (2);  
insert into test1 values (2);  
insert into test1 values (null);
```

ID
1
2
2
-

4 Rows

Download CSV

4 rows selected.

```
create table test2 (id number);
insert into test2 values (1);
insert into test2 values (1);
insert into test2 values (2);
insert into test2 values (2);
insert into test2 values (null);
```


ID
1
1
2
2
-

} 5 records

> If you see the above example , we have 4 records in 1 table test1 as 1,2,2,Null and 5 records in other table test2 as 1,1,2,2,Null

--> How many records will come if we use left join

```

1 select test1.id, test2.id id_2
2 from test1 left join test2 on test1.id = test2.id;

```

ID	ID_2
1	1
1	1
2	2
2	2
2	2
2	2
-	-

Download CSV
7 rows selected.

1. Who is Left?
test 1,
so all from test1 &
matching from test2

- See this solution above. and try to understand line by line.
- Who is left. test1 is left. so all records from test1 will come and only match from test2 will come
- test1 has first record as 1, so 1 will come and against that 1 there is 2 matching record in test2, so 1 come 2 times

1 from test1 <-> First 1 from test2
1 from test1 <--> Second 1 from test2

- So against 1 record of test1, we are getting 2 records from test2 and output has 2 rows
- same go with next 2. In the test1 table we have 2 times "2". For the first 2 we have matching 2 rows in test2 tables. so each 2 will come 2 times.
- Null can not compare so it will be null

```
select * from test1
```

ID
2
2
1
-

→ Null introduce

4 rows selected.

```
select * from test2
```

ID
-
1
1
2
2

5 rows selected.

```
select test1.id id_1, test2.id id_2
from test1 inner join test2 on test1.id = test2.id
```

ID_1	ID_2
1	1
1	1
2	2
2	2
2	2
2	2

6 rows selected.

```
select test1.id id_1, test2.id id_2
from test1 left join test2 on test1.id = test2.id
```

ID_1	ID_2
1	1
1	1
2	2
2	2
2	2
2	2
-	-

7 rows selected.

This null
Present in left side table

This null bcoz no matching

=====

A	B	inner	Left	Right
1	1	1 1	1 1	1 1
1	2	1 1	1 1	1 2 → we have 2's
2	2	2 2	2 2	2 2
2	2	2 2	2 2	2 2
3	2		N N	N N
		↓ only match	N N	N N
			N 2 → 2 line 2	N 2
			2 2	2 2
			3 N → Do not find any thing matching	2 2
			↓ All left side as it is then match	↓ All right side it is match & then start match

C1	C2	Cross	inner	left	Right
1	2	5x5 = 25	1 1	1 1	2 2
2	3		2 2	2 2	N 3
4	1		1 1	4 N	1 1
1	5			1 1	1 5
N	N			N N	N N
5	5				

FULL OUTER JOIN

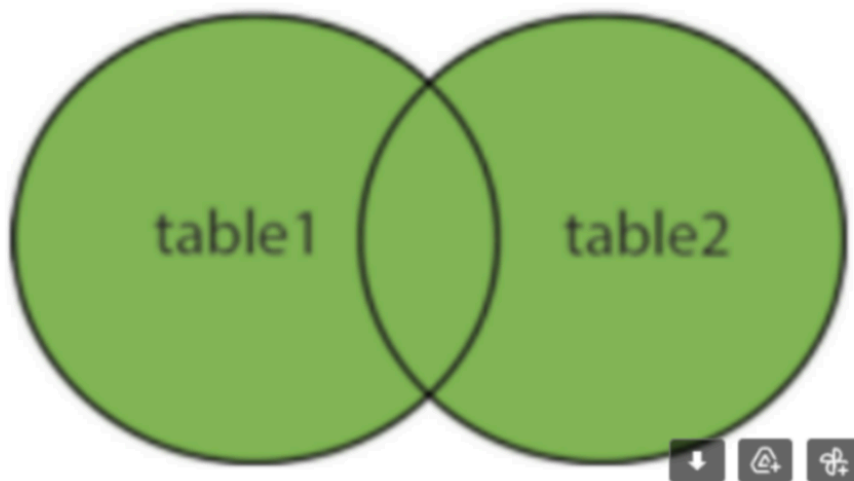
The FULL OUTER JOIN keyword returns all records when there is a match in left (table1) or right (table2) table records.

```

SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;

```

FULL OUTER JOIN



Means Everything. (Matching + Not matching) For example, let's create 2 small table :

```
create table A (id number);
insert into A values (1);
insert into A values (2);
insert into A values (3);
```

```
create table B (id number);
insert into B values (4);
insert into B values (2);
insert into B values (3);
```

A Left B			A Right B		
ID	A	B			
1	1	NULL	NULL	4	
2	2	2	2	2	
3	3	2	3	3	

[Download CSV](#)
3 rows selected.

A Full outer B		
ID	A	B
4		4
2	2	2
3	3	3
	1	NULL
	NULL	4

[Download CSV](#)
3 rows selected.

```

1 select A.id A_Id, B.Id B_id
2 From A Full Join B On A.Id = B.Id

```

A_ID	B_ID
-	4
2	2
3	3
1	-

Download CSV
4 rows selected.

> matching
+
not matching

A	B	INNER	Left	Right	Full
1	3	1 1	1 1	N 3	1 1
1	4	1 1	1 1	N 4	1 1
N	N	2 2	N N	N N	N N
N	N	1 1	N N	N N	N N
2	1		2 2	1 1	2 2
N	2		N N	1 1	N N
1			1 1	2 2	1 1
		4	7	8	2 2 2 2 2 2 2 2 3 4 2 2

<u>A</u>	<u>B</u>	<u>inner</u>		<u>left</u>	
1	1	1	1	1	1
1	N	1	1	1	1
N	N	2	2	N	N
N	2			N	N
N	3			N	N
2	4			2	2
<u>Right</u>		<u>Full outer</u>			
1	1	1	1	1	1
1	N	N	N	N	N
2	2	N	N	N	N
2	3	2	2	2	2
2	4	N	N	N	N
3		N	N	N	N
4		N	N	N	N
				3	3
				4	4

=====

```

create table A (id number);
create table B (id number);
create table C (id number);

```

```

Insert into A values (1);
Insert into A values (2);
Insert into A values (3);
Insert into B values (3);
Insert into B values (4);
Insert into B values (5);
Insert into C values (4);
Insert into C values (5);
Insert into C values (6);
Insert into C values (1);
Insert into C values (2);

```

<u>A</u>	<u>B</u>	<u>C</u>
1	3	4
2	4	5
3	5	6
		1
		2

$3 \times 3 \times 5$
 $= 45$ - Cross join

<u>A</u>	<u>B</u>	<u>C</u>
1	3	4
2	4	5
3	5	6
		1
		2

not a single
 record
 common in
 all 3 tables
 so, 0 record

```

37 select * from
38 A,B,C where A.id = B.id inner join
39 and B.id = C.id
40
  
```

Script Output x Query Result x

SQL | All Rows Fetched: 0 in 0.006 seconds

ID	ID_1	ID_2
----	------	------

→ No Record

A B C

1 3 4
2 4 5
3 5 6
 1
 2

SQL Query:

```

36 select * from
37 A left join B On A.id = B.id
38 Left join C on A.id = C.Id;
39
40

```

Script Output x Query Result x

SQL | All Rows Fetched: 3 in 0.012 seconds

ID	ID_1	ID_2
1	1 (null)	1
2	2 (null)	2
3	3 (null)	3 (null)

Handwritten notes:

- All from left matching from right
- 1 N
2 N
3 3
- +
- 1 1
2 2
3 N

A B C

1 3 4
2 4 5
3 5 6
 1
 2

SQL Query:

```

36 select * from
37 A left join B On A.id = B.id
38 Left join C on B.id = C.Id;
39
40

```

Script Output x Query Result x

SQL | All Rows Fetched: 3 in 0.008 seconds

ID	ID_1	ID_2
1	1 (null)	(null)
2	2 (null)	(null)
3	3 (null)	3 (null)

Handwritten notes:

- All from B & match C
- 1 N
2 N
3 3
- +
- N N
N N
3 N

Handwritten data for tables A, B, and C:

A	B	C
1	3	4
2	9	5
3	5	6
		1
		2

SQL Query:

```

select * from
A right join B On A.id = B.id
Left join C on B.id = C.id;
  
```

Query Result:

ID	ID_1	ID_2
1 (null)	4	4
2 (null)	5	5
3	3	(null)

Handwritten notes: "All from B & match from 1" (pointing to the join condition), "3 3", "N 4", "N 5", "3 N", "4 4", "5 5".

SQL Query:

```

select * from
A left join b on a.id > b.id
  
```

Query Result:

ID	ID_1
1	(null)
2	(null)
3	(null)

Handwritten notes: "All from A", "No one is greater", "1 N", "2 N", "3 N".

Problem Statement:

I have a Order table and I have a customer table. Customer table contains all customer related information and the order table contains all data related to the orders placed by the customer. Write a SQL to get all customer name and their orders.

Solution: Understand here, that what the question wants.

- It wants the customer name and the corresponding order for that customer.
- Customer name from customer table and Order details from Order table
- 2 different tables, so we need 2-1 = 1 minimum join.
- how can we join , both tables must have something in common. In our scenario, it is given that customer_id is common.

```

select CUST_FIRST_NAME, CUST_LAST_NAME, ORDER_TOTAL
from oe.customers , oe.orders
where oe.customers.CUSTOMER_ID = oe.orders.CUSTOMER_ID

```

Customer-id is common

inner join

CUST_FIRST_NAME	CUST_LAST_NAME	ORDER_TOTAL
Sivaji	Landis	62303
Sivaji	Landis	10082.3
Sivaji	Landis	71173
Sivaji	Landis	11188.5
Sivaji	Landis	5537.8

Suppose, question changed and now I want all the records from the customer, who never placed any order .

- What does it mean? It means data exists in the customer table, but not exists corresponding entry in the order table . correct?
- So, we want all records from customer, but they should not present in order table (we will learn better way, but time being just see from join)
- So, we will use left join and put customer table in left, so all data from customer will come and only matching records from order table
- So, if record not found in order, so will get "null"

All records from left side table (customer)

```

select CUST_FIRST_NAME, CUST_LAST_NAME, ORDER_TOTAL
from oe.customers left join oe.orders
on oe.customers.CUSTOMER_ID = oe.orders.CUSTOMER_ID

```

& only matches from right side table (orders)

CUST_FIRST_NAME	CUST_LAST_NAME	ORDER_TOTAL
Sivaji	Landis	62303
Sivaji	Landis	10082.3
Sivaji	Landis	71173
Sivaji	Landis	11188.5

But what we want, where the order entry is null. Order data entry is null means those records not present in orders table

These customers do not
Placed
any
orders

```

select * from (
select CUST_FIRST_NAME, CUST_LAST_NAME, ORDER_TOTAL
from oe.customers left join oe.orders
on oe.customers.CUSTOMER_ID = oe.orders.CUSTOMER_ID
) where ORDER_TOTAL is null

```

→ Add this

CUST_FIRST_NAME	CUST_LAST_NAME	ORDER_TOTAL
Hema	Powell	-
Harry Mean	Peckinpah	-
Blake	Seignier	-
Claude	Powell	-

You can directly write the SQL, without subquery as below:

```

select c.customer_id, c.CUST_FIRST_NAME, c.CUST_LAST_NAME, o.ORDER_ID, o.order_total
from oe.customers c left join oe.orders o
on c.customer_id = o.customer_id
where ORDER_ID is null

```

→ This condition use to filter all such data

CUSTOMER_ID	CUST_FIRST_NAME	CUST_LAST_NAME	ORDER_ID	ORDER_TOTAL
171	Hema	Powell	-	-
172	Harry Mean	Peckinpah	-	-
174	Blake	Seignier	-	-